SWE30011 - IoT Programming

Week 2 Tutorial

Digital Sensors

Objective

In this tutorial, we will explore how to integrate different types of digital sensors with Arduino to read sensor data and trigger actions based on the readings. A digital sensor directly outputs data in a digital format, typically as binary signals (0s and 1s), rather than an analog voltage or current. These binary readings correspond to LOW (0) and HIGH (1) values in Arduino. Digital sensors often include an analog-to-digital converter (ADC) within their circuitry, allowing them to process raw sensor readings and transmit precise, noise-resistant data to a microcontroller. In this tutorial, we will read data from digital sensors and use actuators to perform actions based on the sensor inputs. An actuator is a device that receives an electrical signal from the microcontroller and converts it into a physical action, such as movement, light, or sound. Some common actuators found in an Arduino kit include buzzers, LEDs, DC motors, and servo motors. Figure 1 illustrates some of the digital sensors and actuators commonly used in IoT applications.

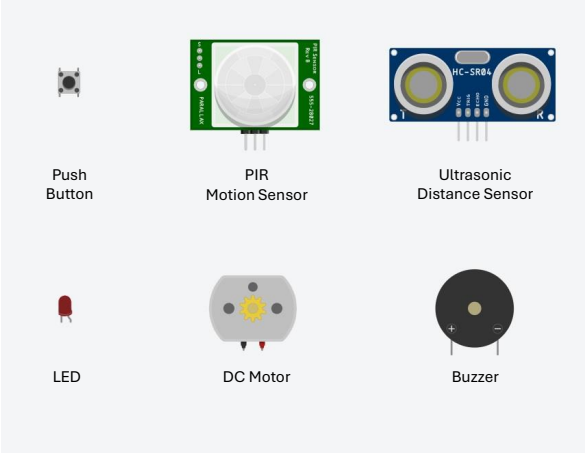


Figure 1: Sample Digital Sensors (Top Row) and Actuators (Bottom Row)

Task 1: Using Push Button to Control a Fan State

In this task, we will use a push button to toggle a fan between on and off states. A push button is a simple digital sensor that detects whether it is pressed or released. When pressed, it closes the circuit, sending either a LOW (0) or HIGH (1) signal to the microcontroller, depending on the circuit configuration. When released, the circuit opens, changing the signal state. Figure 2 illustrates the push button connections at different states.

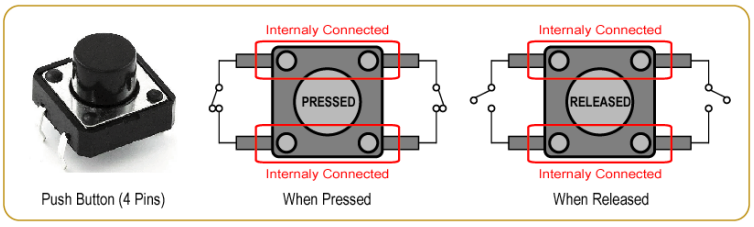


Figure 2: Push Button Connections at Different States

To control the state of a fan using a push button, two independent circuits need to be implemented: one for the fan and one for the push button. The fan's state is then controlled through software when a change in the push button's state is detected. Figure 3 shows the push button circuit used to control the fan's state.

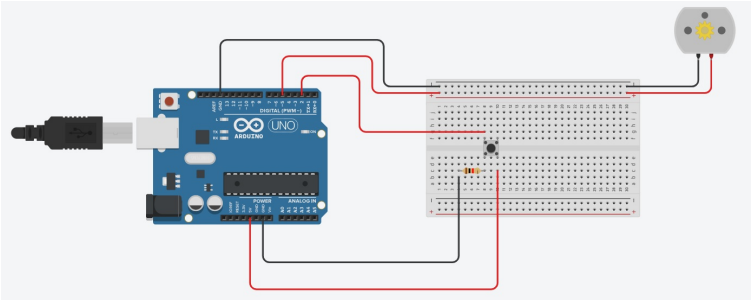


Figure 3: Push Button Circuit to Control a Fan State

Note: In Arduino programming, the constants `HIGH` and `LOW` are used to represent the digital states of `1` and `0`, respectively.

Task 2: Light-based Alarm System using PIR Motion Sensor

In this task, we use a PIR (Passive Infrared) motion sensor to trigger a red LED to turn on when motion is detected. Our objective is to turn on the LED when motion is detected and turn off the LED when no motion is detected. A PIR motion sensor detects motion by measuring changes in infrared radiation. It works based on the principle that all objects emit infrared radiation as a function of their temperature, such as humans and animals. The PIR sensor detects variations in infrared radiation when a person or object moves within its field of view. Figure 4 shows the PIR motion sensor circuit used to implement an LED-based alarm system.

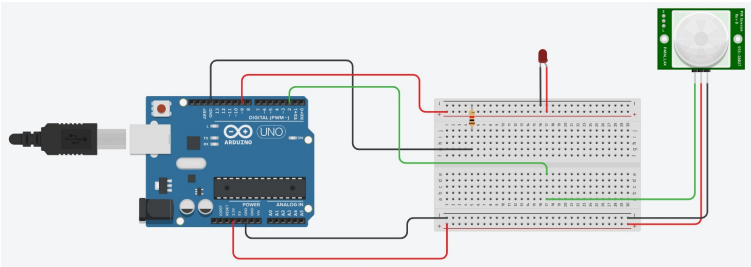


Figure 4: The Circuit of an LED-based Alarm System using PIR Motion Sensor

Task 3: Sound-based Alarm System using Ultrasonic Distance Sensor

In this task, we aim to implement an alarm system that operates based on the distance of an object from the sensor. Our objective is to activate the alarm if an object is closer than 20 centimeters to the sensor, and deactivate it if the object is farther than 20 centimeters. An ultrasonic distance sensor measures the distance between the sensor and an object using sound waves. It works by emitting a high-frequency sound pulse and measuring the time it takes for the sound to bounce back after hitting an object. Figure 5 shows the circuit of the alarm system using the ultrasonic distance sensor.

Congratulations! You are now familiar with using digital sensors in IoT applications.

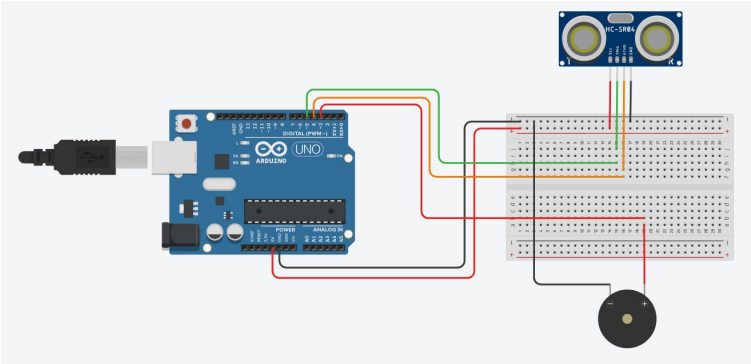


Figure 5: Alarm System Circuit using Ultrasonic Distance Sensor